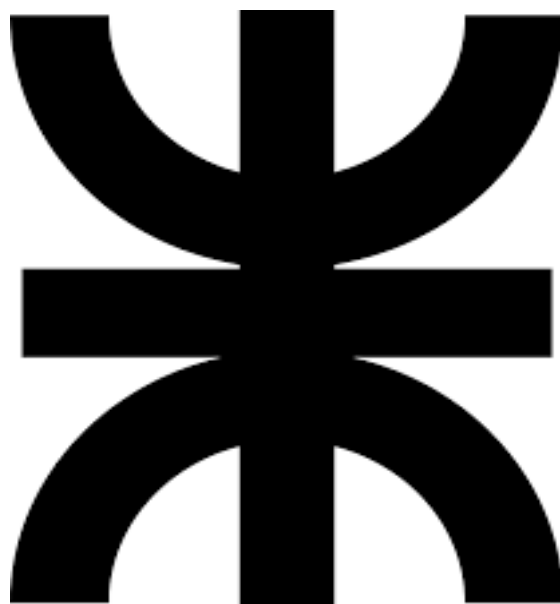


UNIDAD 3 - ACTIVIDAD 3 - BITÁCORA

Release de base de datos v2.0



UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA

Materia: Sistema de Gestión de Base de Datos.

Comisión: K 3.1

Año: 2026

Grupo: 6

Integrantes:

- Basualdo Alvarez, Facundo F.
- Godoy, Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria Eugenia.
- Sanchez, Romina.
- Toledo, Francisco.

Docentes:

- Prof. Andrés Fantin.
- JTP. Fernández, Juan Carlos.
- Aux. Edgar Nahuel Dominguez.

ÍNDICE

Plantilla de bitácora:	3
Problema a resolver:	3
Decisiones técnicas:.....	3
Comandos y evidencias:	4
Pruebas realizadas:	8
Errores y diagnóstico:.....	9
Uso de IA:.....	10
Aprendizajes:	10

Plantilla de bitácora:

Problema a resolver:

Describir brevemente el requerimiento.

El requerimiento consiste en actualizar el esquema de la base de datos de la versión v1 a la v2.0 para incorporar nuevos estados de pedido, una tabla de auditoría para registrar sus cambios, restricciones que impidan precios o totales negativos, y una función para consultar el total acumulado por cliente. El principal desafío técnico es implementar todos estos cambios directamente sobre el entorno en producción sin perder los datos preexistentes, sin recrear tablas desde cero y dejando preparado un plan de contingencia (*rollback*) por si fuera necesario revertir la actualización.

Decisiones técnicas:

Indicar qué construcciones SQL u objetos del motor se eligieron y por qué.

Para implementar los nuevos requerimientos sobre la base en producción sin romper nada ni perder datos, elegimos las siguientes construcciones y objetos en PostgreSQL:

- **Modificación de esquema con ALTER TABLE ... ADD COLUMN ... DEFAULT 'pendiente' NOT NULL:** Porque la base ya tenía pedidos cargados de la v1 (1001, 1002 y 1003). Si agregábamos la columna estado como NOT NULL sin un valor por defecto, la sentencia fallaba por los nulos; y si no la poníamos NOT NULL, nos quedaban datos incompletos. Con DEFAULT 'pendiente', PostgreSQL completó automáticamente los registros viejos sin tener que intervenirlos a mano.
- **Restricciones de verificación (CHECK constraints):** Porque usamos CHECK (estado IN (...)) para los estados permitidos, y CHECK (total >= 0) junto con CHECK (precio_unitario >= 0) para que no entren números negativos. Elegimos resolverlo como restricción declarativa en el motor en lugar de validarlo por código de aplicación para asegurar la regla de negocio directamente en los datos: si entra un dato inválido, el motor lo rebota al instante.
- **Tabla de auditoría con clave foránea e identidad (IDENTITY y REFERENCES):** Porque creamos auditoria_estado_pedido con auditoria_id GENERATED ALWAYS AS IDENTITY para tener una clave primaria autoincremental limpia y estándar de SQL. Además, usamos pedido_id REFERENCES pedido(pedido_id) para mantener integridad

referencial y asegurar que cada registro de auditoría pertenezca a un pedido real. Para el tiempo usamos CURRENT_TIMESTAMP por defecto, así la fecha y hora se guardan solas al momento del cambio.

- **Función PL/pgSQL y Disparador (TRIGGER a nivel de fila):** Porque crear la tabla de auditoría sola no alcanzaba porque quedaba vacía. Por eso creamos una función registrar_estado() disparada por un trigger AFTER UPDATE ON pedido FOR EACH ROW. Evaluamos con old.estado IS DISTINCT FROM new.estado para que el trigger inserte en la auditoría únicamente cuando el estado realmente cambia, evitando registros basura si se actualiza otro campo como el total o la fecha.
- **Función en lenguaje procedural con LEFT JOIN y COALESCE:** Porque para calcular el total acumulado por cliente creamos la función total_cliente(). Inicialmente un INNER JOIN o un GROUP BY sobre pedido dejaba afuera a los clientes sin compras (como el Cliente Uno con ID 3). Elegimos hacer cliente LEFT JOIN pedido junto con COALESCE(SUM(p.total), 0) para que todos los clientes aparezcan en el reporte y aquellos sin compras devuelvan 0 en lugar de un valor nulo o quedar descartados.

Comandos y evidencias:

Pegar comandos relevantes, resultados, planes o capturas referenciadas.

- a. Evidencia: **Persistencia de los datos originales de la V1:** La consigna exige demostrar que la migración no destruyó tuplas preexistentes.

Comando:

```
sgbd_lab=# SELECT * FROM cliente;  
SELECT * FROM pedido;  
SELECT * FROM detalle_pedido;
```

Resultado:

cliente_id	nombre	email
1	Norte SA	norte@example.com
2	Sur SRL	sur@example.com
3	Cliente Uno	

(3 filas)

pedido_id	cliente_id	fecha	total	estado
1001	1	2026-09-01	15000.00	pendiente
1002	1	2026-09-12	22000.00	pendiente
1003	2	2026-09-15	9000.00	pendiente

(3 filas)

detalle_id	pedido_id	producto	cantidad	precio_unitario
1	1001	Servicio Base	1	15000.00
2	1002	Servicio Premium	1	22000.00
3	1003	Servicio Base	1	9000.00

(3 filas)

sgbd_lab=# |

- b. Evidencia: **Validación de estados permitidos y restricciones de dominio:** Para demostrar que el motor rechaza valores ilícitos, primero se agrega la restricción para limitar los estados válidos:

```
sgbd_lab=# ALTER TABLE pedido
ADD CONSTRAINT chk_estado
CHECK (estado IN ('pendiente', 'pagado', 'enviado', 'cancelado'));
ALTER TABLE
sgbd_lab=# |
```

- Ahora forzando los intentos de carga inválidas:

```
sgbd_lab=# UPDATE pedido SET estado = 'bloqueado' WHERE pedido_id = 1002;
UPDATE detalle_pedido SET precio_unitario = -25000 WHERE detalle_id = 2;
```

- El resultado:

```
ERROR: el nuevo registro para la relación «pedido» viola la restricción «check» «chk_estado»
DETALLE: La fila que falla contiene (1002, 1, 2026-09-12, 22000.00, bloqueado).
ERROR: el nuevo registro para la relación «detalle_pedido» viola la restricción «check» «chk_precio»
DETALLE: La fila que falla contiene (2, 1002, Servicio Premium, 1, -25000.00).
sgbd_lab=# |
```

- c. Evidencia: **Automatización y funcionamiento de la tabla de auditoría:** Como la tabla de auditoría por sí sola no registra nada de forma pasiva, definimos la función y el disparador automático:

```
sgbd_lab=# CREATE OR REPLACE FUNCTION registrar_estado()
RETURNS TRIGGER AS $$
BEGIN
    IF old.estado IS DISTINCT FROM new.estado THEN
        INSERT INTO auditoria_estado_pedido (pedido_id, estado_ant, estado_nuev)
        VALUES (new.pedido_id, old.estado, new.estado);
    END IF;
    RETURN new;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_estado
AFTER UPDATE ON pedido
FOR EACH ROW EXECUTE FUNCTION registrar_estado();
CREATE FUNCTION
CREATE TRIGGER
sgbd_lab=# |
```

- Ejecutando los cambios de estado válidos:

```
sgbd_lab=# UPDATE pedido SET estado = 'enviado' WHERE pedido_id = 1001;
UPDATE pedido SET estado = 'cancelado' WHERE pedido_id = 1003;
sgbd_lab=# |
```

- Consultamos el contenido de la tabla de auditoria: SELECT * FROM auditoria_estado_pedido;
- Resultado:

```
sgbd_lab=# SELECT * FROM auditoria_estado_pedido;
 auditoria_id | pedido_id | estado_ant | estado_nuev |          fecha_cambio
-----+-----+-----+-----+-----
          1 |      1001 | pendiente | enviado      | 2026-09-13 18:20:00.962969
          2 |      1003 | pendiente | cancelado    | 2026-09-13 18:20:00.974299
(2 filas)

sgbd_lab=# |
```

- d. Evidencia: **Función reutilizable de totales por cliente:** Cargamos la versión final corregida con LEFT JOIN y COALESCE:

```
sgbd_lab=# CREATE OR REPLACE FUNCTION total_cliente()
RETURNS TABLE (id_cliente INTEGER, total_cliente NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT c.cliente_id, COALESCE(SUM(p.total), 0)
    FROM cliente c LEFT JOIN pedido p ON c.cliente_id = p.cliente_id
    GROUP BY c.cliente_id
    ORDER BY c.cliente_id ASC;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
sgbd_lab=# |
```

- Consultando la función con: `SELECT * FROM total_cliente();`
- Resultado:

```
sgbd_lab=# SELECT * FROM total_cliente();
 id_cliente | total_cliente
-----+-----
          1 |      37000.00
          2 |       9000.00
          3 |           0
(3 filas)

sgbd_lab=# |
```

e. **Contingencia y reversibilidad (Rollback)**

- Comandos del script de reversion (06_rollback_v2.sql):

```
-- 1. Eliminar triggers y funciones
DROP TRIGGER IF EXISTS trg_auditoria_estado ON pedido;
DROP FUNCTION IF EXISTS registrar_estado();
DROP FUNCTION IF EXISTS total_cliente();

-- 2. Eliminar restricciones agregadas
ALTER TABLE detalle_pedido DROP CONSTRAINT IF EXISTS chk_precio;
ALTER TABLE pedido DROP CONSTRAINT IF EXISTS chk_total;
ALTER TABLE pedido DROP CONSTRAINT IF EXISTS chk_estado;

-- 3. Eliminar la tabla nueva
DROP TABLE IF EXISTS auditoria_estado_pedido;

-- 4. Revertir la tabla pedido al esquema de la v1
ALTER TABLE pedido DROP COLUMN IF EXISTS estado;
```

- Resultados: Al ejecutar ese script tiene el siguiente impacto:

- Al eliminar la relación auditoria_estado_pedido, se destruye irreversiblemente el registro cronológico de los cambios de estado.
- Al quitar la columna estado de la tabla pedido, se pierde la información operativa de qué pedidos estaban pagados, enviados o cancelados

Pruebas realizadas:

- **Caso válido:** Actualizar el estado de un pedido existente a uno de los valores permitidos ('enviado').
- Comando: UPDATE pedido SET estado = 'enviado' WHERE pedido_id = 1001;
- Resultado:

```
sgbd_lab=# UPDATE pedido SET estado = 'enviado' WHERE pedido_id = 1001;
UPDATE 1
sgbd_lab=# |
```

La sentencia se ejecuta con éxito (UPDATE 1). La restricción chk_estado valida el valor y el disparador trg_auditoria_estado registra automáticamente en la tabla auditoria_estado_pedido el cambio de 'pendiente' a 'enviado' junto con su marca temporal.

- **Caso inválido:** Intentar registrar un monto financiero negativo (violación de dominio) o un estado no catalogado.
- Comando: UPDATE pedido SET total = -1000 WHERE pedido_id = 1002;
- Resultado: Esto comprueba que ninguna tupla inconsistente puede persistir en la base de datos.

```
sgbd_lab=# UPDATE pedido SET total = -1000 WHERE pedido_id = 1002;
ERROR:  el nuevo registro para la relación «pedido» viola la restricción «check» «chk_total»
DETALLE:  La fila que falla contiene (1002, 1, 2026-09-12, -1000.00, pendiente).
sgbd_lab=# |
```

- **Caso límite:** Consultar el acumulado de compras para un cliente que no tiene ningún pedido registrado en la base de datos (Cliente 3: "Cliente Uno" con ID 3).
- Comando: SELECT * FROM total_cliente() WHERE id_cliente = 3;
- Resultado: Gracias al uso de LEFT JOIN y COALESCE(SUM(p.total), 0), la función no ignora al cliente ni devuelve un valor indeterminado (NULL). Retorna formalmente la fila con valor 0.

```
sgbd_lab=# SELECT * FROM total_cliente() WHERE id_cliente = 3;
 id_cliente | total_cliente 
-----+-----
          3 |              0
(1 fila)
sgbd_lab=#
```

Errores y diagnóstico:

Qué falló, cómo se detectó y cómo se corrigió.

Durante el desarrollo e implementación de la versión 2.0 nos encontramos con:

- Inactividad de la tabla de auditoría:
 - *Qué falló:* Al crear inicialmente la tabla auditoria_estado_pedido, notamos que los cambios de estado en pedido no se registraban.
 - *Cómo se detectó:* Tras hacer varios UPDATE de prueba sobre los pedidos, ejecutamos SELECT * FROM auditoria_estado_pedido; y la tabla seguía vacía.
 - *Cómo se corrigió:* Diagnosticamos que el motor no tiene un mecanismo de auditoría automático por defecto. Lo resolvimos programando en PL/pgSQL la función registrar_estado() y asociándola al trigger trg_auditoria_estado con el evento AFTER UPDATE ON pedido FOR EACH ROW.
- Omisión de clientes sin compras en la función de cálculo:
 - *Qué falló:* La primera versión de la función total_cliente() calculaba la suma agrupando directamente sobre la tabla pedido.
 - *Cómo se detectó:* Al consultar SELECT * FROM total_cliente();, solo devolvía filas para los clientes 1 y 2, dejando totalmente afuera al cliente 3 ("Cliente Uno").
 - *Cómo se corrigió:* Cambiamos la consulta interna utilizando un LEFT JOIN partiendo desde la tabla cliente, y envolvimos la suma con COALESCE(SUM(p.total), 0). De este modo, los clientes sin pedidos no se descartan y muestran un acumulado explícito de 0.
- Error de conexión inicial por base de datos inexistente en el entorno local:
 - *Qué falló:* Al intentar conectarnos a la base con el metacomando \c sgbd_lab, la terminal arrojó el error FATAL: no existe la base de datos «sgbd_lab».

- *Cómo se detectó:* Por el mensaje explícito devuelto en consola al arrancar la sesión de psql en la máquina virtual de Multipass (mv-sgbd).
- *Cómo se corrigió:* Se diagnosticó que en la instancia local de la VM aún no se había inicializado la base de datos. Se ejecutó CREATE DATABASE sgbd_lab; desde la sesión administrativa y posteriormente se aplicó el script base de la v1.

Uso de IA:

Indicar si se usó IA, para qué, qué se validó y qué se descartó

- **Para qué se utilizó:** Se empleó como asistente interactivo para la sintaxis precisa de PL/pgSQL (disparadores y funciones que devuelven tablas), para estructurar el plan de pruebas mínimas y para recordar comandos de gestión en la terminal de Linux y PostgreSQL.
- **Qué se validó:** Se validó cada bloque de código sugerido ejecutándolo directamente en la consola interactiva psql de nuestra máquina virtual. Se verificó el diccionario de datos con \d auditoria_estado_pedido, se comprobaron las violaciones forzadas de las restricciones CHECK y se contrastaron los cálculos de totales acumulados contra los valores conocidos de la v1.
- **Qué se descartó:** Se descartaron propuestas de triggers complejos basados en sentencias STATEMENT en lugar de FOR EACH ROW (que no capturaban adecuadamente los valores OLD y NEW por fila individual), y se rechazó la idea de recrear tablas o utilizar valores nulos para el nuevo campo de estado, manteniendo la restricción estricta de no perder datos en la migración

Aprendizajes:

Qué queda consolidado para la próxima actividad

- **Evolución no destructiva de esquemas:** Comprendimos en la práctica cómo realizar modificaciones de estructura sobre tablas pobladas usando ALTER TABLE ... ADD COLUMN ... DEFAULT ... NOT NULL, garantizando retrocompatibilidad e integridad sin requerir reinicializar la base.

- **Integridad forzada en el motor:** Consolidamos el valor de las restricciones CHECK (chk_total, chk_precio, chk_estado) como primera línea de defensa para el dominio de datos, comprobando que el SGBD actúa de barrera automática ante datos erróneos antes de que se persistan.
- **Programabilidad reactiva y consistencia en reportes:** Aprendimos a vincular funciones PL/pgSQL y disparadores para automatizar pistas de auditoría sin intervención del usuario, y reafirmamos el uso de LEFT JOIN junto con COALESCE para evitar sesgos o pérdidas de entidades en agregaciones y consultas analíticas.
- **Planificación de contingencia (Rollback):** Nos quedó claro que cualquier script de evolución en producción debe acompañarse de su contraparte de reversión, documentando explícitamente qué datos son irrecuperables (como el historial de auditoría o los estados avanzados) en caso de tener que volver a la versión anterior.